

Packet Capture & Network Traffic Analysis (tcpdump)

Executive Summary

This lab demonstrated my ability to capture and analyse live network traffic using **tcpdump** on a Linux system. Network packet analysis is a core skill in SOC and incident response roles. It enables analysts to identify suspicious activity, investigate incidents, and understand attacker behaviour at the network level.

Key Tasks Completed

- Identified network interfaces using `ifconfig` and `tcpdump -D`
- Captured and analysed live packets on `eth0`, interpreting TCP flags, TTL, TOS and protocol details
- Captured HTTP traffic to a pcap file using port filtering and background processing
- Analysed captured packets using `-r`, `-v` and `-X` flags for detailed inspection

Tools Used: tcpdump, Linux CLI, Bash, curl

Packet capturing and analysing

In this lab activity, I performed tasks associated with using **tcpdump** to capture network traffic. I was asked to capture data in a packet capture (p-cap) file and then examine the contents of the captured packet data to focus on specific types of traffic.

Scenario

I acted as a network analyst who needed to use **tcpdump** to capture and analyse live network traffic from a Linux virtual machine.

Task 1. Identify network interfaces

At the beginning of the lab, my task was to identify the network interfaces that could be used to capture network packet data. To do this, I used the command *sudo ifconfig*.

The command returned the following output:

```
analyst@3366a0da4d0c:~$ sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1460
    inet 172.17.0.2 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:ac:11:00:02 txqueuelen 0 (Ethernet)
    RX packets 799 bytes 14005282 (13.3 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 460 bytes 40858 (39.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 89 bytes 11840 (11.5 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 89 bytes 11840 (11.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Here, I used **eth0** as the interface to capture data. Next, I ran the command with **-D** to identify the interface options for packet capture as some of the system may not include the **ifconfig** command. The output looked like the following:

```
analyst@3366a0da4d0c:~$ sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1460
    inet 172.17.0.2 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:ac:11:00:02 txqueuelen 0 (Ethernet)
    RX packets 799 bytes 14005282 (13.3 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 460 bytes 40858 (39.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 89 bytes 11840 (11.5 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 89 bytes 11840 (11.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

analyst@3366a0da4d0c:~$ sudo tcpdump -D
1.eth0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
5.nflog (Linux netfilter log (NFLOG) interface) [none]
6.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
7.dbus-system (D-Bus system bus) [none]
8.dbus-session (D-Bus session bus) [none]
analyst@3366a0da4d0c:~$
```

Task 2. Inspect the network traffic of a network interface with tcpdump

I used tcpdump command `sudo tcpdump -i eth0 -v -c5` to filter live network packet traffic on **eth0** interface where `-i eth0` captured data specifically from eth0 interface, `-v` displayed detailed packet data and `-c5` captured 5 packets of data.

```
analyst@3366a0da4d0c:~$ sudo tcpdump -i eth0 -v -c 5
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length
262144 bytes
14:45:43.971211 IP (tos 0x0, ttl 64, id 6081, offset 0, flags [DF], prot
o TCP (6), length 127)
    3366a0da4d0c.5000 > nginx-us-west1-c.c.qwiklabs-terminal-vms-prod-00
.internal.39152: Flags [P.], cksum 0x5924 (incorrect -> 0x28d2), seq 387
9466317:3879466392, ack 4286736144, win 998, options [nop,nop,TS val 369
8422490 ecr 3612794260], length 75
14:45:43.971560 IP (tos 0x0, ttl 63, id 42381, offset 0, flags [DF], pro
to TCP (6), length 52)
    nginx-us-west1-c.c.qwiklabs-terminal-vms-prod-00.internal.39152 > 33
66a0da4d0c.5000: Flags [.], cksum 0x27e1 (correct), ack 75, win 507, opt
ions [nop,nop,TS val 3612794282 ecr 3698422490], length 0
14:45:44.074201 IP (tos 0x0, ttl 64, id 6082, offset 0, flags [DF], prot
o TCP (6), length 142)
    3366a0da4d0c.5000 > nginx-us-west1-c.c.qwiklabs-terminal-vms-prod-00
.internal.39152: Flags [P.], cksum 0x5933 (incorrect -> 0xde0c), seq 75:
165, ack 1, win 998, options [nop,nop,TS val 3698422593 ecr 3612794282],
length 90
14:45:44.074556 IP (tos 0x0, ttl 63, id 42382, offset 0, flags [DF], pro
to TCP (6), length 52)
    nginx-us-west1-c.c.qwiklabs-terminal-vms-prod-00.internal.39152 > 33
66a0da4d0c.5000: Flags [.], cksum 0x26b9 (correct), ack 165, win 507, op
tions [nop,nop,TS val 3612794385 ecr 3698422593], length 0
14:45:44.089380 IP (tos 0x0, ttl 64, id 50853, offset 0, flags [DF], pro
to UDP (17), length 71)
    3366a0da4d0c.33588 > metadata.google.internal.domain: 51439+ PTR? 13
7.0.22.172.in-addr.arpa. (43)
```

In the above capture, I could see that tcpdump was listening on the **eth0, link-type EN10MB(Ethernet)**. Next line showed **timestamp** 14:45:43.971211 IP, followed by the protocol type, IP. Then, `-v` provided more details about the IP packet fields, such as TOS, TTL, offset, flags, internal protocol type (e.g., TCP (6)), as well as the length of the outer packet in bytes. The following section showed the communication from **internal.39152** to **3366a0da4d0c.5000**. Following that, the packet identified TCP flags; flag '**P**' represented **push** and the period indicated it was an ACK flag, meaning that the packet was pushing out data.

Task 3. Capture network traffic with tcpdump

1) I captured data into a file called **capture.pcap** using the command `sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap &` and pressed **Enter**.

```
analyst@3366a0da4d0c:~$ sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.
pcap &
[1] 13577
analyst@3366a0da4d0c:~$ tcpdump: listening on eth0, link-type EN10MB (Et
hernet), snapshot length 262144 bytes
```

Here, I told **tcpdump** to capture 9 packets of data (-c9) from interface **eth0**, but not to resolve IP addresses or ports to names using **-nn**, which wouldn't inform the attacker about the capture, and I also indicated that it should filter only port 80. Then, I asked it to save (-w) the captured data to the file **capture.pcap**. Finally, I instructed the 'Bash' shell to run the command in the background by using '&'.

2) Next, I used 'curl' command to generate some HTTP (port 80) traffic.

```
analyst@3366a0da4d0c:~$ curl opensource.google.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="https://opensource.google/">here</A>.
</BODY></HTML>
analyst@3366a0da4d0c:~$ 9 packets captured
10 packets received by filter
0 packets dropped by kernel
ls -l capture.pcap
-rw-r--r-- 1 tcpdump tcpdump 1412 Feb 16 15:00 capture.pcap
[1]+  Done                  sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap
analyst@3366a0da4d0c:~$
```

In a later step, I verified that the packet had been captured by using the **ls -l** command, and then I filtered the captured packet data.

```
analyst@3366a0da4d0c:~$ sudo tcpdump -nn -r capture.pcap -v
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:00:26.712800 IP (tos 0x0, ttl 64, id 21145, offset 0, flags [DF], proto TCP (6), length 60)
    172.17.0.2.48398 > 172.253.117.139.80: Flags [S], cksum 0xceca (incorrect -> 0x1052), seq 1001589279, win 65320, options [mss 1420, sackOK, TS val 172776597 ecr 0, nop, wscale 6], length 0
15:00:26.713305 IP (tos 0x0, ttl 126, id 0, offset 0, flags [DF], proto TCP (6), length 60)
    172.253.117.139.80 > 172.17.0.2.48398: Flags [S.], cksum 0xf2a8 (correct), seq 4229887069, ack 1001589280, win 65535, options [mss 1420, sackOK, TS val 2910682820 ecr 172776597, nop, wscale 8], length 0
15:00:26.713317 IP (tos 0x0, ttl 64, id 21146, offset 0, flags [DF], proto TCP (6), length 52)
    172.17.0.2.48398 > 172.253.117.139.80: Flags [.], cksum 0xcec2 (incorrect -> 0x1d50), ack 1, win 1021, options [nop, nop, TS val 172776598 ecr 2910682820], length 0
15:00:26.713356 IP (tos 0x0, ttl 64, id 21147, offset 0, flags [DF], proto TCP (6), length 137)
    172.17.0.2.48398 > 172.253.117.139.80: Flags [P.], cksum 0xcf17 (incorrect -> 0x8b03), seq 1:86, ack 1, win 1021, options [nop, nop, TS val 172776598 ecr 2910682820], length 85: HTTP, length: 85
    GET / HTTP/1.1
    Host: opensource.google.com
    User-Agent: curl/7.74.0
    Accept: */*
```

Here, **-r** was used for reading the captured data and **-v** for viewing detailed packet data.

Finally, I used **-X** for extended packet data, as requested by the lab.

```
analyst@3366a0da4d0c:~$ sudo tcpdump -nn -r capture.pcap -X
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:00:26.712800 IP 172.17.0.2.48398 > 172.253.117.139.80: Flags [S], seq 1001589279, win 65320, options [mss 1420,sackOK,TS val 172776597 ecr 0,nop,wscale 6], length 0
    0x0000: 4500 003c 5299 4000 4006 1987 ac11 0002  E..<R.@@.....
..
    0x0010: acfd 758b bd0e 0050 3bb3 0a1f 0000 0000  ..u....P;.....
..
    0x0020: a002 ff28 ceca 0000 0204 058c 0402 080a  ...(.
..
    0x0030: 0a4c 5c95 0000 0000 0103 0306          .L\.....
15:00:26.713305 IP 172.253.117.139.80 > 172.17.0.2.48398: Flags [S.], seq 4229887069, ack 1001589280, win 65535, options [mss 1420,sackOK,TS val 2910682820 ecr 172776597,nop,wscale 8], length 0
    0x0000: 4500 003c 0000 4000 7e06 2e20 acfd 758b  E..<..@.~.....
u.
    0x0010: ac11 0002 0050 bd0e fc1e f45d 3bb3 0a20  ....P.....];.
..
    0x0020: a012 ffff f2a8 0000 0204 058c 0402 080a  .....
..
    0x0030: ad7d 7ec4 0a4c 5c95 0103 0308          .}~..L\.....
15:00:26.713317 IP 172.17.0.2.48398 > 172.253.117.139.80: Flags [.), ack 1, win 1021, options [nop,nop,TS val 172776598 ecr 2910682820], length 0
    0x0000: 4500 0034 529a 4000 4006 198e ac11 0002  E..4R.@@.....
```

Lessons Learned & Conclusion

This lab reinforced the importance of precise packet filtering in a SOC environment. Using **'-nn'** prevented DNS resolution during the capture, which reduced noise and avoided alerting potential attackers. Background processing with **'&'** allowed for simultaneous traffic generation and capture, while **'-X'** revealed hex and ASCII payload data useful for identifying malicious content. Overall, this lab strengthened my practical network forensics skills, which are highly applicable to incident investigation and threat detection.